🎓 Bloom's Taxonomy LLM Fine-Tuning Pipeline

Task: Fine-tune three instruction-tuned LLMs to classify academic learning outcomes into Bloom's Taxonomy cognitive levels.

Taxonomy Levels: Remember | Understand | Apply | Analyze | Evaluate | Create

---

#	Model	Parameters	Fine-Tuning Method
1	`mistralai/Mistral-7B-Instruct-v0.3`	7B	QLoRA (4-bit)
2	`Qwen/Qwen2-7B-Instruct`	7B	QLoRA (4-bit)
3	`google/gemma-2-9b-it`	9B	QLoRA (4-bit)

Hardware: NVIDIA RTX A6000 (48GB VRAM)
Dataset: `sample_full.csv` — 21,379 labelled learning outcomes
Chunked Loading: ✅ Memory-safe dataset streaming in configurable chunks

---

> 📌 **One-Click Run:** Execute all cells top-to-bottom. Each model section is fully self-contained.

0 · CUDA Diagnostic & Environment Fix

In [3]:
```
# CUDA FALSE DIAGNOSTIC & FIX FOR RTX A6000 (FIXED VERSION)
# This version avoids importlib.reload() issues

import subprocess
import sys
import os

print("="*60)
print("CUDA DIAGNOSTIC & FIX")
print("="*60)

# 1. Check nvidia-smi
print("\n[1] nvidia-smi output:")
result = subprocess.run(["nvidia-smi"], capture_output=True, text=True)
print(result.stdout if result.returncode == 0 else "ERROR: nvidia-smi failed")

# 2. Check PyTorch CUDA info
print("\n[2] PyTorch build info:")
import torch
print(f"PyTorch version: {torch.__version__}")
print(f"CUDA used to build PyTorch: {torch.version.cuda}")
```

```
print(f"CUDA available: {torch.cuda.is_available()}")

# 3. Check for common issues
print("\n[3] Environment check:")
print(f"CUDA_HOME: {os.environ.get('CUDA_HOME', 'NOT SET')}")
print(f"LD_LIBRARY_PATH: {os.environ.get('LD_LIBRARY_PATH', 'NOT SET')[:80]}...")

# 4. Check if CPU-only version was installed
print("\n[4] Checking if CPU-only PyTorch was installed...")
result = subprocess.run([sys.executable, "-m", "pip", "show", "torch"], capture_output=True, te
if "cpu" in result.stdout.lower():
    print("WARNING: CPU-only PyTorch detected!")
    needs_reinstall = True
else:
    print("PyTorch package info OK")
    needs_reinstall = False

# 5. Check NVIDIA driver version
print("\n[5] Driver check:")
try:
    result = subprocess.run(["nvidia-smi", "--query-gpu=driver_version", "--format=csv,noheader
                             capture_output=True, text=True)
    driver_ver = result.stdout.strip()
    print(f"Driver version: {driver_ver}")
except:
    print("Could not detect driver version")

# 6. FIX: Force reinstall with correct CUDA version
if not torch.cuda.is_available() or needs_reinstall:
    print("\n" + "="*60)
    print("FIX REQUIRED: Reinstalling PyTorch with CUDA support...")
    print("="*60)

    print("\nStep A: Uninstalling current PyTorch...")
    subprocess.run([sys.executable, "-m", "pip", "uninstall", "torch", "torchvision", "torchau
                    capture_output=True)

    print("Step B: Installing PyTorch with CUDA 12.4...")
    result = subprocess.run([
        sys.executable, "-m", "pip", "install",
        "torch==2.6.0", "torchvision==0.21.0", "torchaudio==2.6.0",
        "--index-url", "https://download.pytorch.org/whl/cu124",
        "--force-reinstall", "--no-cache-dir"
    ], capture_output=True, text=True)

    if result.returncode != 0:
        print(f"Installation error: {result.stderr}")
    else:
        print("Installation completed successfully!")

    print("\n" + "="*60)
    print("IMPORTANT: You MUST restart your Jupyter kernel/terminal now!")
    print("="*60)
    print("\nAfter restart, run this to verify:")
    print("import torch")
```

```
    print("print(torch.cuda.is_available())")
    print("print(torch.cuda.get_device_name(0))")

    with open('/tmp/verify_cuda.py', 'w') as f:
        f.write("""import torch
print("="*60)
print("CUDA Verification")
print("="*60)
print(f"PyTorch: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"VRAM: {torch.cuda.get_device_properties(0).total_memory / 1024**3:.1f} GB")
    print("SUCCESS! RTX A6000 is ready.")
else:
    print("FAILED - GPU not detected")
""")
    print("\nVerification script saved to: /tmp/verify_cuda.py")
    print("Run it after restart: python /tmp/verify_cuda.py")

else:
    print("\n[6] No fix needed - CUDA is already working!")

print("="*60)
print("RESTART KERNEL NOW...")

import torch
print(torch.cuda.is_available())
print(torch.cuda.get_device_name(0))
```

Out[3]:

```
============================================================
CUDA DIAGNOSTIC & FIX
============================================================

[1] nvidia-smi output:
Tue Apr  7 10:08:56 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 550.144.03             Driver Version: 550.144.03     CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA RTX A6000               Off |   00000000:00:05.0 Off |                  Off |
| 30%   34C    P8             24W /  300W |       4MiB /  49140MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                              GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
```

```
+-----------------------------------------------------------------------------------------+


[2] PyTorch build info:
PyTorch version: 2.1.1+cu121
CUDA used to build PyTorch: 12.1
CUDA available: True

[3] Environment check:
CUDA_HOME: NOT SET
LD_LIBRARY_PATH: /usr/local/cuda/lib64:...

[4] Checking if CPU-only PyTorch was installed...
PyTorch package info OK

[5] Driver check:
Driver version: 550.144.03

[6] No fix needed - CUDA is already working!
============================================================
RESTART KERNEL NOW...
True
NVIDIA RTX A6000
```

1 · Install Dependencies

In [4]:

```python
import subprocess, sys

# Install all dependencies (rich is required by trl>=0.8)
pkgs = [
    "rich",                         # trl hard dependency
    "transformers==4.44.2",
    "peft==0.12.0",
    "trl==0.10.1",
    "bitsandbytes==0.43.3",
    "accelerate==0.34.2",
    "datasets==2.21.0",
    "scikit-learn",
    "pandas",
    "numpy",
    "huggingface_hub",
    "sentencepiece",
    "protobuf",
]

result = subprocess.run(
    [sys.executable, "-m", "pip", "install", "-q"] + pkgs,
    capture_output=True, text=True
)
if result.returncode != 0:
    print("⚠  pip errors:")
    print(result.stderr[-2000:])
```

```
else:
    print("✅ All dependencies installed successfully.")
```

Out[4]:

```
✅ All dependencies installed successfully.
```

2 · HuggingFace Login

In [5]:

```
import huggingface_hub
from huggingface_hub import login

login("hf_byvpkLIEiQdmtaXnvSRdYseblNHbXtnPs")
print("✅ Login Successful")
```

Out[5]:

```
✅ Login Successful
```

3 · Global Imports & Configuration

In [6]:

```
import os
import gc
import json
import warnings
import numpy as np
import pandas as pd
import torch

from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    BitsAndBytesConfig,
    TrainingArguments,
    pipeline,
)
from peft import LoraConfig, get_peft_model, TaskType, PeftModel
from trl import SFTTrainer
from sklearn.metrics import (
    classification_report,
    accuracy_score,
    f1_score,
    confusion_matrix,
)
from sklearn.model_selection import train_test_split

warnings.filterwarnings("ignore")

# ── Reproducibility ──────────────────────────────────────
SEED = 42
torch.manual_seed(SEED)
```

```
np.random.seed(SEED)

# ── Dataset path ──────────────────────────────────────────────
DATASET_PATH  = "content/sample_full.csv"
CHUNK_SIZE    = 2000       # rows processed per chunk during data loading

# ── Bloom's Taxonomy label set ────────────────────────────────
BLOOM_LABELS  = ["Remember", "Understand", "Apply", "Analyze", "Evaluate", "Create"]

# ── Training hyper-parameters (shared across models) ──────────
MAX_SEQ_LEN   = 256
NUM_EPOCHS    = 3
BATCH_SIZE    = 4
GRAD_ACCUM    = 4          # effective batch = 16
LEARNING_RATE = 2e-4
TEST_SIZE     = 0.15       # 15% held-out test split

# ── QLoRA configuration ───────────────────────────────────────
LORA_R        = 16
LORA_ALPHA    = 32
LORA_DROPOUT  = 0.05

device = "cuda" if torch.cuda.is_available() else "cpu"
print(f"✅ Using device : {device}")
if device == "cuda":
    print(f"   GPU          : {torch.cuda.get_device_name(0)}")
    print(f"   VRAM         : {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
```

Out[6]:

```
2026-04-07 10:09:11.285454: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:9261]
Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one
has already been registered
2026-04-07 10:09:11.285519: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:607]
Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one
has already been registered
2026-04-07 10:09:11.292400: E external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1515]
Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one
has already been registered
2026-04-07 10:09:11.318303: I tensorflow/core/platform/cpu_feature_guard.cc:182] This
TensorFlow binary is optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with
the appropriate compiler flags.
2026-04-07 10:09:12.563557: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT
Warning: Could not find TensorRT
```

Out[6]:

```
[2026-04-07 10:09:15,407] [INFO] [real_accelerator.py:158:get_accelerator] Setting
ds_accelerator to cuda (auto detect)
✅ Using device : cuda
   GPU          : NVIDIA RTX A6000
   VRAM         : 51.0 GB
```

4 · Dataset Loading (Chunk-by-Chunk) & Preprocessing

In [7]:

```
# ──────────────────────────────────────────────────────────────
#  Load CSV in memory-safe chunks, then assemble a clean
#  single-label DataFrame ready for supervised fine-tuning.
# ──────────────────────────────────────────────────────────────

def load_dataset_chunked(path: str, chunk_size: int = 2000) -> pd.DataFrame:
    """Read CSV chunk-by-chunk, return assembled single-label DataFrame."""
    chunks = []
    total_rows = 0
    dropped    = 0

    reader = pd.read_csv(path, chunksize=chunk_size)
    for i, chunk in enumerate(reader):
        chunk.columns = chunk.columns.str.strip()
        chunk["Learning_outcome"] = chunk["Learning_outcome"].str.strip()

        # Build a single label from the one-hot columns
        label_cols = [c for c in BLOOM_LABELS if c in chunk.columns]
        chunk[label_cols] = chunk[label_cols].apply(pd.to_numeric, errors="coerce")

        # Keep only rows with exactly one positive label
        label_sum = chunk[label_cols].notna().sum(axis=1)
        valid_mask = label_sum == 1
        dropped += (~valid_mask).sum()

        chunk = chunk[valid_mask].copy()

        # Collapse one-hot → string label
        chunk["label"] = chunk[label_cols].apply(
            lambda row: label_cols[row.notna().values.argmax()], axis=1
        )

        chunks.append(chunk[["Learning_outcome", "label"]])
        total_rows += len(chunk)
        print(f"  Chunk {i+1:>3} loaded — running total: {total_rows:,} valid rows", end="\r")

    print()
    df = pd.concat(chunks, ignore_index=True)
    df = df.dropna(subset=["Learning_outcome", "label"])
    df = df[df["Learning_outcome"].str.len() > 10]   # remove stub entries
    return df, dropped


print(f"Loading dataset from: {DATASET_PATH}")
print(f"Chunk size           : {CHUNK_SIZE:,} rows\n")

df_full, n_dropped = load_dataset_chunked(DATASET_PATH, CHUNK_SIZE)

print(f"\n{'─'*50}")
print(f"Total usable rows : {len(df_full):,}")
print(f"Rows dropped      : {n_dropped:,}  (multi-label or unlabelled)")
```

```
print(f"\nLabel distribution:")
print(df_full["label"].value_counts().to_string())
```

Out[7]:

```
Loading dataset from: content/sample_full.csv
Chunk size           : 2,000 rows

  Chunk  11 loaded — running total: 18,773 valid rows

──────────────────────────────────────────────────
Total usable rows : 18,773
Rows dropped      : 2,607  (multi-label or unlabelled)

Label distribution:
label
Understand    5079
Apply         5074
Create        2955
Evaluate      2468
Analyze       2311
Remember       886
```

In [8]:

```
# ──────────────────────────────────────────────────────────────
#  Build prompt format + stratified train/test split
# ──────────────────────────────────────────────────────────────

SYSTEM_PROMPT = (
    "You are an expert educator. Classify the following academic learning outcome "
    "into exactly one of Bloom's Taxonomy cognitive levels: "
    "Remember, Understand, Apply, Analyze, Evaluate, or Create. "
    "Respond with only the level name."
)

def make_prompt(learning_outcome: str, label: str = None) -> str:
    """Build an instruction-style prompt. Include label for training, omit for inference."""
    prompt = (
        f"### System:\n{SYSTEM_PROMPT}\n\n"
        f"### Learning Outcome:\n{learning_outcome.strip()}\n\n"
        f"### Bloom's Level:"
    )
    if label is not None:
        prompt += f" {label}"
    return prompt


df_full["text"] = df_full.apply(
    lambda r: make_prompt(r["Learning_outcome"], r["label"]), axis=1
)

# Stratified split to preserve label balance
df_train, df_test = train_test_split(
    df_full,
    test_size=TEST_SIZE,
    stratify=df_full["label"],
```

```
    random_state=SEED,
)

print(f"Train rows : {len(df_train):,}")
print(f"Test  rows : {len(df_test):,}")
print("\nSample prompt:")
print("—" * 60)
print(df_full["text"].iloc[0])
```

Out[8]:

```
Train rows : 15,957
Test  rows : 2,816

Sample prompt:
————————————————————————————————————————————————————————————

### System:
You are an expert educator. Classify the following academic learning outcome into exactly one
of Bloom's Taxonomy cognitive levels: Remember, Understand, Apply, Analyze, Evaluate, or
Create. Respond with only the level name.

### Learning Outcome:
Analyze the health economic implications of efficient perioperative practice.

### Bloom's Level: Analyze
```

In [9]:

```
# ——————————————————————————————————————————————————————————
#  Shared helper: evaluate a fine-tuned model on the test set
# ——————————————————————————————————————————————————————————

def evaluate_model(model, tokenizer, df_test: pd.DataFrame, model_label: str):
    """
    Run greedy inference on df_test, extract predicted label from
    generated text, and print a full classification report.
    """
    model.eval()
    predictions, ground_truth = [], []

    gen_pipeline = pipeline(
        "text-generation",
        model=model,
        tokenizer=tokenizer,
        max_new_tokens=8,
        do_sample=False,
        device_map="auto",
        pad_token_id=tokenizer.eos_token_id,
    )

    print(f"\nRunning inference on {len(df_test):,} test samples…")
    batch_size = 16
    prompts    = [make_prompt(lo) for lo in df_test["Learning_outcome"]]
    labels     = df_test["label"].tolist()

    for i in range(0, len(prompts), batch_size):
        batch  = prompts[i : i + batch_size]
```

```
        outputs = gen_pipeline(batch, batch_size=batch_size)

        for out, gold in zip(outputs, labels[i : i + batch_size]):
            generated = out[0]["generated_text"].split("### Bloom's Level:")[-1].strip()
            # Extract first word that matches a Bloom label
            pred = "Unknown"
            for word in generated.split():
                if word.capitalize() in BLOOM_LABELS:
                    pred = word.capitalize()
                    break
            predictions.append(pred)
            ground_truth.append(gold)

        if (i // batch_size + 1) % 10 == 0:
            print(f"  Processed {min(i + batch_size, len(prompts)):,} / {len(prompts):,}", end=

    print()

    # ── Metrics ──────────────────────────────────────────────
    acc   = accuracy_score(ground_truth, predictions)
    f1_w  = f1_score(ground_truth, predictions, average="weighted",  zero_division=0)
    f1_m  = f1_score(ground_truth, predictions, average="macro",     zero_division=0)

    print(f"\n{'='*60}")
    print(f"  📊  Evaluation Results — {model_label}")
    print(f"{'='*60}")
    print(f"  Accuracy         : {acc:.4f}")
    print(f"  Weighted F1      : {f1_w:.4f}")
    print(f"  Macro F1         : {f1_m:.4f}")
    print(f"{'-'*60}")
    print(classification_report(
        ground_truth, predictions, labels=BLOOM_LABELS, zero_division=0
    ))

    results = {
        "model"       : model_label,
        "accuracy"    : round(acc,  4),
        "f1_weighted" : round(f1_w, 4),
        "f1_macro"    : round(f1_m, 4),
        "predictions" : predictions,
        "ground_truth": ground_truth,
    }
    return results


# Storage for cross-model comparison
ALL_RESULTS = {}
print("✅ Evaluation helper defined.")
```

Out[9]:

```
✅ Evaluation helper defined.
```

5 · Model 1 — `mistralai/Mistral-7B-Instruct-v0.3`

QLoRA fine-tuning with 4-bit NF4 quantisation, LoRA rank 16, 3 epochs.

In [10]:
```
# ── Configuration ──────────────────────────────────────────
MODEL_1_NAME   = "mistralai/Mistral-7B-Instruct-v0.3"
MODEL_1_OUTDIR = "./mistral_bloom_finetuned"
MODEL_1_LABEL  = "Mistral-7B-Instruct-v0.3"

print(f"Model : {MODEL_1_NAME}")
print(f"Output: {MODEL_1_OUTDIR}")
```

Out[10]:
```
Model : mistralai/Mistral-7B-Instruct-v0.3
Output: ./mistral_bloom_finetuned
```

In [11]:
```
# ── 5.1  Load tokenizer ────────────────────────────────────
tokenizer_1 = AutoTokenizer.from_pretrained(
    MODEL_1_NAME,
    trust_remote_code=True,
)
tokenizer_1.pad_token    = tokenizer_1.eos_token
tokenizer_1.padding_side = "left"
print(f"✅ Tokenizer loaded. Vocab size: {tokenizer_1.vocab_size:,}")
```

Out[11]:
```
tokenizer_config.json: 0.00B [00:00, ?B/s]
```

Out[11]:
```
tokenizer.model:   0%|          | 0.00/587k [00:00<?, ?B/s]
```

Out[11]:
```
tokenizer.json: 0.00B [00:00, ?B/s]
```

Out[11]:
```
special_tokens_map.json:   0%|          | 0.00/414 [00:00<?, ?B/s]
```

Out[11]:
```
✅ Tokenizer loaded. Vocab size: 32,768
```

In [12]:
```
# ── 5.2  Load model in 4-bit (QLoRA) ──────────────────────
bnb_config_1 = BitsAndBytesConfig(
    load_in_4bit              = True,
    bnb_4bit_use_double_quant = True,
    bnb_4bit_quant_type       = "nf4",
    bnb_4bit_compute_dtype    = torch.bfloat16,
)

model_1 = AutoModelForCausalLM.from_pretrained(
    MODEL_1_NAME,
    quantization_config  = bnb_config_1,
    device_map           = "auto",
    trust_remote_code    = True,
```

```
)
model_1.config.use_cache = False
print(f"✅ Base model loaded on: {device}")
```

Out[12]:
```
config.json:   0%|          | 0.00/601 [00:00<?, ?B/s]
```

Out[12]:
```
model.safetensors.index.json: 0.00B [00:00, ?B/s]
```

Out[12]:
```
Downloading shards:   0%|          | 0/3 [00:00<?, ?it/s]
```

Out[12]:
```
model-00001-of-00003.safetensors:   0%|          | 0.00/4.95G [00:00<?, ?B/s]
```

Out[12]:
```
model-00002-of-00003.safetensors:   0%|          | 0.00/5.00G [00:00<?, ?B/s]
```

Out[12]:
```
model-00003-of-00003.safetensors:   0%|          | 0.00/4.55G [00:00<?, ?B/s]
```

Out[12]:
```
Loading checkpoint shards:   0%|          | 0/3 [00:00<?, ?it/s]
```

Out[12]:
```
generation_config.json:   0%|          | 0.00/116 [00:00<?, ?B/s]
```

Out[12]:
```
✅ Base model loaded on: cuda
```

In [13]:
```
# ── 5.3  Attach LoRA adapter ──────────────────────────────
lora_config_1 = LoraConfig(
    r              = LORA_R,
    lora_alpha     = LORA_ALPHA,
    lora_dropout   = LORA_DROPOUT,
    bias           = "none",
    task_type      = TaskType.CAUSAL_LM,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"],
)

model_1 = get_peft_model(model_1, lora_config_1)
model_1.print_trainable_parameters()
```

Out[13]:
```
trainable params: 41,943,040 || all params: 7,289,966,592 || trainable%: 0.5754
```

In [14]:
```
# ── 5.4  Prepare HuggingFace datasets ─────────────────────
train_dataset_1 = Dataset.from_pandas(df_train[["text"]].reset_index(drop=True))
test_dataset_1  = Dataset.from_pandas(df_test[["text"]].reset_index(drop=True))

print(f"Training samples : {len(train_dataset_1):,}")
print(f"Test samples     : {len(test_dataset_1):,}")
```

```
Training samples : 15,957
Test samples     : 2,816
```

In [15]:

```python
# ── 5.5  TrainingArguments ──────────────────────────────
training_args_1 = TrainingArguments(
    output_dir                  = MODEL_1_OUTDIR,
    num_train_epochs            = NUM_EPOCHS,
    per_device_train_batch_size = BATCH_SIZE,
    gradient_accumulation_steps = GRAD_ACCUM,
    learning_rate               = LEARNING_RATE,
    lr_scheduler_type           = "cosine",
    warmup_ratio                = 0.05,
    fp16                        = False,
    bf16                        = True,
    logging_steps               = 50,
    evaluation_strategy         = "epoch",
    save_strategy               = "epoch",
    load_best_model_at_end      = True,
    metric_for_best_model       = "eval_loss",
    report_to                   = "none",
    seed                        = SEED,
    dataloader_num_workers      = 2,
    group_by_length             = True,
)
```

In [16]:

```python
# ── 5.6  SFTTrainer & train ─────────────────────────────
trainer_1 = SFTTrainer(
    model           = model_1,
    args            = training_args_1,
    train_dataset   = train_dataset_1,
    eval_dataset    = test_dataset_1,
    tokenizer       = tokenizer_1,
    dataset_text_field = "text",
    max_seq_length  = MAX_SEQ_LEN,
    packing         = False,
)

print(f"{'='*60}")
print(f"  🚀  Fine-tuning {MODEL_1_LABEL}")
print(f"{'='*60}")
trainer_1.train()
```

Out[16]:

```
Map:   0%|          | 0/15957 [00:00<?, ? examples/s]
```

Out[16]:

```
Map:   0%|          | 0/2816 [00:00<?, ? examples/s]
```

Out[16]:

```
============================================================
  🚀  Fine-tuning Mistral-7B-Instruct-v0.3
============================================================
```

```
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
```

Out[16]: [2991/2991 1:51:30, Epoch 2/3]

Epoch	Training Loss	Validation Loss
0	0.567600	0.612186
2	0.236400	0.614046

Out[16]:

```
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
```

```
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
huggingface/tokenizers: The current process just got forked, after parallelism has already
been used. Disabling parallelism to avoid deadlocks...
To disable this warning, you can either:
        - Avoid using `tokenizers` before the fork if possible
        - Explicitly set the environment variable TOKENIZERS_PARALLELISM=(true | false)
```

Out[16]:

```
TrainOutput(global_step=2991, training_loss=0.430093581921199, metrics={'train_runtime': 6693.9
```

In [17]:

```
# ── 5.7  Save adapter ──────────────────────────────────────
trainer_1.save_model(MODEL_1_OUTDIR)
tokenizer_1.save_pretrained(MODEL_1_OUTDIR)
print(f"✅ Model saved to: {MODEL_1_OUTDIR}")
```

Out[17]:

```
✅ Model saved to: ./mistral_bloom_finetuned
```

In [18]:

```
# ── 5.8  Evaluation ────────────────────────────────────────
results_1 = evaluate_model(model_1, tokenizer_1, df_test, MODEL_1_LABEL)
ALL_RESULTS[MODEL_1_LABEL] = results_1
```

Out[18]:

```
The model 'PeftModelForCausalLM' is not supported for text-generation. Supported models are
['BartForCausalLM', 'BertLMHeadModel', 'BertGenerationDecoder', 'BigBirdForCausalLM',
'BigBirdPegasusForCausalLM', 'BioGptForCausalLM', 'BlenderbotForCausalLM',
'BlenderbotSmallForCausalLM', 'BloomForCausalLM', 'CamembertForCausalLM', 'LlamaForCausalLM',
'CodeGenForCausalLM', 'CohereForCausalLM', 'CpmAntForCausalLM', 'CTRLLMHeadModel',
'Data2VecTextForCausalLM', 'DbrxForCausalLM', 'ElectraForCausalLM', 'ErnieForCausalLM',
'FalconForCausalLM', 'FuyuForCausalLM', 'GemmaForCausalLM', 'Gemma2ForCausalLM',
'GitForCausalLM', 'GPT2LMHeadModel', 'GPT2LMHeadModel', 'GPTBigCodeForCausalLM',
'GPTNeoForCausalLM', 'GPTNeoXForCausalLM', 'GPTNeoXJapaneseForCausalLM', 'GPTJForCausalLM',
'JambaForCausalLM', 'JetMoeForCausalLM', 'LlamaForCausalLM', 'MambaForCausalLM',
'Mamba2ForCausalLM', 'MarianForCausalLM', 'MBartForCausalLM', 'MegaForCausalLM',
'MegatronBertForCausalLM', 'MistralForCausalLM', 'MixtralForCausalLM', 'MptForCausalLM',
'MusicgenForCausalLM', 'MusicgenMelodyForCausalLM', 'MvpForCausalLM', 'NemotronForCausalLM',
'OlmoForCausalLM', 'OpenLlamaForCausalLM', 'OpenAIGPTLMHeadModel', 'OPTForCausalLM',
'PegasusForCausalLM', 'PersimmonForCausalLM', 'PhiForCausalLM', 'Phi3ForCausalLM',
'PLBartForCausalLM', 'ProphetNetForCausalLM', 'QDQBertLMHeadModel', 'Qwen2ForCausalLM',
'Qwen2MoeForCausalLM', 'RecurrentGemmaForCausalLM', 'ReformerModelWithLMHead',
'RemBertForCausalLM', 'RobertaForCausalLM', 'RobertaPreLayerNormForCausalLM',
'RoCBertForCausalLM', 'RoFormerForCausalLM', 'RwkvForCausalLM', 'Speech2Text2ForCausalLM',
'StableLmForCausalLM', 'Starcoder2ForCausalLM', 'TransfoXLLMHeadModel', 'TrOCRForCausalLM',
'WhisperForCausalLM', 'XGLMForCausalLM', 'XLMWithLMHeadModel', 'XLMProphetNetForCausalLM',
'XLMRobertaForCausalLM', 'XLMRobertaXLForCausalLM', 'XLNetLMHeadModel', 'XmodForCausalLM'].
```

Out[18]: Running inference on 2,816 test samples…

Out[18]: You seem to be using the pipelines sequentially on GPU. In order to maximize efficiency please use a dataset

Out[18]: Processed 2,720 / 2,816

```
═══════════════════════════════════════════════════════════
  📊  Evaluation Results — Mistral-7B-Instruct-v0.3
═══════════════════════════════════════════════════════════
  Accuracy         : 0.9322
  Weighted F1      : 0.9324
  Macro F1         : 0.9206
───────────────────────────────────────────────────────────
```

	precision	recall	f1-score	support
Remember	0.85	0.88	0.86	133
Understand	0.96	0.94	0.95	762
Apply	0.94	0.93	0.94	761
Analyze	0.92	0.92	0.92	347
Evaluate	0.95	0.91	0.93	370
Create	0.88	0.95	0.92	443
accuracy			0.93	2816
macro avg	0.92	0.92	0.92	2816
weighted avg	0.93	0.93	0.93	2816

In [19]:
```python
# ── 5.9  Free VRAM before loading next model ──────────────────
del model_1, trainer_1, tokenizer_1
gc.collect()
torch.cuda.empty_cache()
print(f"✅ VRAM freed. Available: {torch.cuda.memory_reserved(0)/1e9:.1f} GB reserved")
```

Out[19]: ✅ VRAM freed. Available: 0.3 GB reserved

---

6 · Model 2 — `Qwen/Qwen2-7B-Instruct`

QLoRA fine-tuning with 4-bit NF4 quantisation, LoRA rank 16, 3 epochs.

In [20]:
```python
# ── Configuration ─────────────────────────────────────────────
MODEL_2_NAME   = "Qwen/Qwen2-7B-Instruct"
MODEL_2_OUTDIR = "./qwen2_bloom_finetuned"
MODEL_2_LABEL  = "Qwen2-7B-Instruct"

print(f"Model : {MODEL_2_NAME}")
print(f"Output: {MODEL_2_OUTDIR}")
```

Out[20]:
```
Model : Qwen/Qwen2-7B-Instruct
Output: ./qwen2_bloom_finetuned
```

In [21]:
```
# ── 6.1  Load tokenizer ──────────────────────────────────
tokenizer_2 = AutoTokenizer.from_pretrained(
    MODEL_2_NAME,
    trust_remote_code=True,
)
tokenizer_2.pad_token    = tokenizer_2.eos_token
tokenizer_2.padding_side = "left"
print(f"✅ Tokenizer loaded. Vocab size: {tokenizer_2.vocab_size:,}")
```

Out[21]:
```
tokenizer_config.json: 0.00B [00:00, ?B/s]
```

Out[21]:
```
vocab.json: 0.00B [00:00, ?B/s]
```

Out[21]:
```
merges.txt: 0.00B [00:00, ?B/s]
```

Out[21]:
```
tokenizer.json: 0.00B [00:00, ?B/s]
```

Out[21]:
```
✅ Tokenizer loaded. Vocab size: 151,643
```

In [22]:
```
# ── 6.2  Load model in 4-bit (QLoRA) ─────────────────────
bnb_config_2 = BitsAndBytesConfig(
    load_in_4bit               = True,
    bnb_4bit_use_double_quant  = True,
    bnb_4bit_quant_type        = "nf4",
    bnb_4bit_compute_dtype     = torch.bfloat16,
)

model_2 = AutoModelForCausalLM.from_pretrained(
    MODEL_2_NAME,
    quantization_config  = bnb_config_2,
    device_map           = "auto",
    trust_remote_code    = True,
)
model_2.config.use_cache = False
print(f"✅ Base model loaded on: {device}")
```

Out[22]:
```
config.json:   0%|          | 0.00/663 [00:00<?, ?B/s]
```

Out[22]:
```
model.safetensors.index.json: 0.00B [00:00, ?B/s]
```

Out[22]:
```
Downloading shards:   0%|          | 0/4 [00:00<?, ?it/s]
```

Out[22]:

model-00001-of-00004.safetensors: 0%| | 0.00/3.95G [00:00<?, ?B/s]

Out[22]:

model-00002-of-00004.safetensors: 0%| | 0.00/3.86G [00:00<?, ?B/s]

Out[22]:

model-00003-of-00004.safetensors: 0%| | 0.00/3.86G [00:00<?, ?B/s]

Out[22]:

model-00004-of-00004.safetensors: 0%| | 0.00/3.56G [00:00<?, ?B/s]

Out[22]:

Loading checkpoint shards: 0%| | 0/4 [00:00<?, ?it/s]

Out[22]:

generation_config.json: 0%| | 0.00/243 [00:00<?, ?B/s]

Out[22]:

✅ Base model loaded on: cuda

In [23]:

```
# ── 6.3  Attach LoRA adapter ──────────────────────────────
lora_config_2 = LoraConfig(
    r              = LORA_R,
    lora_alpha     = LORA_ALPHA,
    lora_dropout   = LORA_DROPOUT,
    bias           = "none",
    task_type      = TaskType.CAUSAL_LM,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"],
)

model_2 = get_peft_model(model_2, lora_config_2)
model_2.print_trainable_parameters()
```

Out[23]:

```
trainable params: 40,370,176 || all params: 7,655,986,688 || trainable%: 0.5273
```

In [24]:

```
# ── 6.4  Prepare datasets ─────────────────────────────────
train_dataset_2 = Dataset.from_pandas(df_train[["text"]].reset_index(drop=True))
test_dataset_2  = Dataset.from_pandas(df_test[["text"]].reset_index(drop=True))
print(f"Training samples: {len(train_dataset_2):,}  |  Test: {len(test_dataset_2):,}")
```

Out[24]:

```
Training samples: 15,957  |  Test: 2,816
```

In [25]:

```
# ── 6.5  TrainingArguments ────────────────────────────────
training_args_2 = TrainingArguments(
    output_dir                  = MODEL_2_OUTDIR,
    num_train_epochs            = NUM_EPOCHS,
    per_device_train_batch_size = BATCH_SIZE,
    gradient_accumulation_steps = GRAD_ACCUM,
    learning_rate               = LEARNING_RATE,
    lr_scheduler_type           = "cosine",
    warmup_ratio                = 0.05,
```

```
    fp16                        = False,
    bf16                        = True,
    logging_steps               = 50,
    evaluation_strategy         = "epoch",
    save_strategy               = "epoch",
    load_best_model_at_end      = True,
    metric_for_best_model       = "eval_loss",
    report_to                   = "none",
    seed                        = SEED,
    dataloader_num_workers      = 2,
    group_by_length             = True,
)
```

In [26]:

```
# ── 6.6  SFTTrainer & train ──────────────────────────────
trainer_2 = SFTTrainer(
    model               = model_2,
    args                = training_args_2,
    train_dataset       = train_dataset_2,
    eval_dataset        = test_dataset_2,
    tokenizer           = tokenizer_2,
    dataset_text_field = "text",
    max_seq_length      = MAX_SEQ_LEN,
    packing             = False,
)

print(f"{'='*60}")
print(f"  🚀  Fine-tuning {MODEL_2_LABEL}")
print(f"{'='*60}")
trainer_2.train()
```

Out[26]:

```
Map:   0%|          | 0/15957 [00:00<?, ? examples/s]
```

Out[26]:

```
Map:   0%|          | 0/2816 [00:00<?, ? examples/s]
```

Out[26]:

```
============================================================
  🚀  Fine-tuning Qwen2-7B-Instruct
============================================================
```

Out[26]: [2991/2991 1:38:08, Epoch 2/3]

Epoch	Training Loss	Validation Loss
0	0.691900	0.726392
2	0.393900	0.722525

Out[26]:

```
TrainOutput(global_step=2991, training_loss=0.5824900725660257, metrics={'train_runtime': 5891.
```

In [27]:

```
# ── 6.7  Save adapter ────────────────────────────────────
trainer_2.save_model(MODEL_2_OUTDIR)
tokenizer_2.save_pretrained(MODEL_2_OUTDIR)
```

```
print(f"✅ Model saved to: {MODEL_2_OUTDIR}")
```

Out[27]:

✅ Model saved to: ./qwen2_bloom_finetuned

In [28]:

```
# ── 6.8  Evaluation ──────────────────────────────────────────
results_2 = evaluate_model(model_2, tokenizer_2, df_test, MODEL_2_LABEL)
ALL_RESULTS[MODEL_2_LABEL] = results_2
```

Out[28]:

```
The model 'PeftModelForCausalLM' is not supported for text-generation. Supported models are
['BartForCausalLM', 'BertLMHeadModel', 'BertGenerationDecoder', 'BigBirdForCausalLM',
'BigBirdPegasusForCausalLM', 'BioGptForCausalLM', 'BlenderbotForCausalLM',
'BlenderbotSmallForCausalLM', 'BloomForCausalLM', 'CamembertForCausalLM', 'LlamaForCausalLM',
'CodeGenForCausalLM', 'CohereForCausalLM', 'CpmAntForCausalLM', 'CTRLLMHeadModel',
'Data2VecTextForCausalLM', 'DbrxForCausalLM', 'ElectraForCausalLM', 'ErnieForCausalLM',
'FalconForCausalLM', 'FuyuForCausalLM', 'GemmaForCausalLM', 'Gemma2ForCausalLM',
'GitForCausalLM', 'GPT2LMHeadModel', 'GPT2LMHeadModel', 'GPTBigCodeForCausalLM',
'GPTNeoForCausalLM', 'GPTNeoXForCausalLM', 'GPTNeoXJapaneseForCausalLM', 'GPTJForCausalLM',
'JambaForCausalLM', 'JetMoeForCausalLM', 'LlamaForCausalLM', 'MambaForCausalLM',
'Mamba2ForCausalLM', 'MarianForCausalLM', 'MBartForCausalLM', 'MegaForCausalLM',
'MegatronBertForCausalLM', 'MistralForCausalLM', 'MixtralForCausalLM', 'MptForCausalLM',
'MusicgenForCausalLM', 'MusicgenMelodyForCausalLM', 'MvpForCausalLM', 'NemotronForCausalLM',
'OlmoForCausalLM', 'OpenLlamaForCausalLM', 'OpenAIGPTLMHeadModel', 'OPTForCausalLM',
'PegasusForCausalLM', 'PersimmonForCausalLM', 'PhiForCausalLM', 'Phi3ForCausalLM',
'PLBartForCausalLM', 'ProphetNetForCausalLM', 'QDQBertLMHeadModel', 'Qwen2ForCausalLM',
'Qwen2MoeForCausalLM', 'RecurrentGemmaForCausalLM', 'ReformerModelWithLMHead',
'RemBertForCausalLM', 'RobertaForCausalLM', 'RobertaPreLayerNormForCausalLM',
'RoCBertForCausalLM', 'RoFormerForCausalLM', 'RwkvForCausalLM', 'Speech2Text2ForCausalLM',
'StableLmForCausalLM', 'Starcoder2ForCausalLM', 'TransfoXLLMHeadModel', 'TrOCRForCausalLM',
'WhisperForCausalLM', 'XGLMForCausalLM', 'XLMWithLMHeadModel', 'XLMProphetNetForCausalLM',
'XLMRobertaForCausalLM', 'XLMRobertaXLForCausalLM', 'XLNetLMHeadModel', 'XmodForCausalLM'].
```

Out[28]:

```
Running inference on 2,816 test samples…
  Processed 2,720 / 2,816

═══════════════════════════════════════════════════════════
  📊  Evaluation Results — Qwen2-7B-Instruct
═══════════════════════════════════════════════════════════
  Accuracy         : 0.9247
  Weighted F1      : 0.9250
  Macro F1         : 0.9149
───────────────────────────────────────────────────────────
              precision    recall  f1-score   support

    Remember       0.84      0.89      0.86       133
  Understand       0.97      0.91      0.94       762
       Apply       0.92      0.94      0.93       761
     Analyze       0.89      0.92      0.91       347
    Evaluate       0.97      0.91      0.93       370
      Create       0.88      0.95      0.92       443

    accuracy                           0.92      2816
```

```
   macro avg       0.91      0.92      0.91      2816
weighted avg       0.93      0.92      0.93      2816
```

In [29]:
```
# ── 6.9  Free VRAM before loading next model ──────────────────
del model_2, trainer_2, tokenizer_2
gc.collect()
torch.cuda.empty_cache()
print(f"✅ VRAM freed. Available: {torch.cuda.memory_reserved(0)/1e9:.1f} GB reserved")
```

Out[29]:
```
✅ VRAM freed. Available: 0.3 GB reserved
```

---

7 · Model 3 — `google/gemma-2-9b-it`

QLoRA fine-tuning with 4-bit NF4 quantisation, LoRA rank 16, 3 epochs.

In [30]:
```
# ── Configuration ─────────────────────────────────────────────
MODEL_3_NAME   = "google/gemma-2-9b-it"
MODEL_3_OUTDIR = "./gemma2_bloom_finetuned"
MODEL_3_LABEL  = "Gemma-2-9B-IT"

print(f"Model : {MODEL_3_NAME}")
print(f"Output: {MODEL_3_OUTDIR}")
```

Out[30]:
```
Model : google/gemma-2-9b-it
Output: ./gemma2_bloom_finetuned
```

In []:
```
# ── 7.1  Load tokenizer ──────────────────────────────────────
tokenizer_3 = AutoTokenizer.from_pretrained(
    MODEL_3_NAME,
    trust_remote_code=True,
)
tokenizer_3.pad_token    = tokenizer_3.eos_token
tokenizer_3.padding_side = "left"
print(f"✅ Tokenizer loaded. Vocab size: {tokenizer_3.vocab_size:,}")
```

Out[]:
```
tokenizer_config.json: 0.00B [00:00, ?B/s]
```

Out[]:
```
tokenizer.model:   0%|          | 0.00/4.24M [00:00<?, ?B/s]
```

Out[]:
```
tokenizer.json:   0%|          | 0.00/17.5M [00:00<?, ?B/s]
```

Out[]:
```
special_tokens_map.json:   0%|          | 0.00/636 [00:00<?, ?B/s]
```

Out[]:

✅ Tokenizer loaded. Vocab size: 256,000

In []:
```
# ── 7.2  Load model in 4-bit (QLoRA) ──────────────────────
bnb_config_3 = BitsAndBytesConfig(
    load_in_4bit               = True,
    bnb_4bit_use_double_quant  = True,
    bnb_4bit_quant_type        = "nf4",
    bnb_4bit_compute_dtype     = torch.bfloat16,
)

model_3 = AutoModelForCausalLM.from_pretrained(
    MODEL_3_NAME,
    torch_dtype=torch.bfloat16,          # make sure this is bfloat16, not float16
    attn_implementation="eager",          # ← add this
    device_map="auto",
)
model_3.config.use_cache = False
print(f"✅ Base model loaded on: {device}")
```

Out[]:
```
config.json:   0%|          | 0.00/857 [00:00<?, ?B/s]
```

Out[]:
```
model.safetensors.index.json: 0.00B [00:00, ?B/s]
```

Out[]:
```
Downloading shards:   0%|          | 0/4 [00:00<?, ?it/s]
```

Out[]:
```
model-00001-of-00004.safetensors:   0%|          | 0.00/4.90G [00:00<?, ?B/s]
```

Out[]:
```
model-00002-of-00004.safetensors:   0%|          | 0.00/4.95G [00:00<?, ?B/s]
```

Out[]:
```
model-00003-of-00004.safetensors:   0%|          | 0.00/4.96G [00:00<?, ?B/s]
```

Out[]:
```
model-00004-of-00004.safetensors:   0%|          | 0.00/3.67G [00:00<?, ?B/s]
```

Out[]:
```
Loading checkpoint shards:   0%|          | 0/4 [00:00<?, ?it/s]
```

Out[]:
```
generation_config.json:   0%|          | 0.00/173 [00:00<?, ?B/s]
```

Out[]:
```
✅ Base model loaded on: cuda
```

In []:
```
# Verify dtype after loading
print(next(model_3.parameters()).dtype)  # should be torch.bfloat16
```

Out[]:
```
torch.bfloat16
```

In []:
```
# ── 7.3  Attach LoRA adapter ──────────────────────────────
# Gemma-2 uses different attention projection names
lora_config_3 = LoraConfig(
    r              = LORA_R,
    lora_alpha     = LORA_ALPHA,
    lora_dropout   = LORA_DROPOUT,
    bias           = "none",
    task_type      = TaskType.CAUSAL_LM,
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"],
)

model_3 = get_peft_model(model_3, lora_config_3)
model_3.print_trainable_parameters()
```

Out[]:
```
trainable params: 54,018,048 || all params: 9,295,724,032 || trainable%: 0.5811
```

In []:
```
# ── 7.4  Prepare datasets ─────────────────────────────────
train_dataset_3 = Dataset.from_pandas(df_train[["text"]].reset_index(drop=True))
test_dataset_3  = Dataset.from_pandas(df_test[["text"]].reset_index(drop=True))
print(f"Training samples: {len(train_dataset_3):,}  |  Test: {len(test_dataset_3):,}")
```

Out[]:
```
Training samples: 15,957  |  Test: 2,816
```

In []:
```
# ── 7.5  TrainingArguments ────────────────────────────────
training_args_3 = TrainingArguments(
    output_dir                  = MODEL_3_OUTDIR,
    num_train_epochs            = NUM_EPOCHS,
    per_device_train_batch_size = BATCH_SIZE,
    gradient_accumulation_steps = GRAD_ACCUM,
    learning_rate               = LEARNING_RATE,
    lr_scheduler_type           = "cosine",
    warmup_ratio                = 0.05,
    fp16                        = False,
    bf16                        = True,
    bf16_full_eval              = True,
    logging_steps               = 50,
    evaluation_strategy         = "epoch",
    save_strategy               = "epoch",
    load_best_model_at_end      = True,
    metric_for_best_model       = "eval_loss",
    report_to                   = "none",
    seed                        = SEED,
    dataloader_num_workers      = 2,
    group_by_length             = True,

)
```

In []:
```
# ── 7.6  SFTTrainer & train ───────────────────────────────
trainer_3 = SFTTrainer(
```

```
    model              = model_3,
    args               = training_args_3,
    train_dataset      = train_dataset_3,
    eval_dataset       = test_dataset_3,
    tokenizer          = tokenizer_3,
    dataset_text_field = "text",
    max_seq_length     = MAX_SEQ_LEN,
    packing            = False,
)

print(f"{'='*60}")
print(f"  🚀  Fine-tuning {MODEL_3_LABEL}")
print(f"{'='*60}")
trainer_3.train()
```

Out[]:

```
Map:   0%|          | 0/15957 [00:00<?, ? examples/s]
```

Out[]:

```
Map:   0%|          | 0/2816 [00:00<?, ? examples/s]
```

Out[]:

```
============================================================
  🚀  Fine-tuning Gemma-2-9B-IT
============================================================
```

Out[]:

[2991/2991 1:11:20, Epoch 2/3]

Epoch	Training Loss	Validation Loss
0	0.651700	0.666324
2	0.283300	0.788578

Out[]:

```
TrainOutput(global_step=2991, training_loss=0.5099883208662241, metrics={'train_runtime': 4283.
```

In []:

```
# ── 7.7  Save adapter ──────────────────────────────────
trainer_3.save_model(MODEL_3_OUTDIR)
tokenizer_3.save_pretrained(MODEL_3_OUTDIR)
print(f"✅ Model saved to: {MODEL_3_OUTDIR}")
```

Out[]:

```
✅ Model saved to: ./gemma2_bloom_finetuned
```

In []:

```
# ── 7.8  Evaluation ────────────────────────────────────
results_3 = evaluate_model(model_3, tokenizer_3, df_test, MODEL_3_LABEL)
ALL_RESULTS[MODEL_3_LABEL] = results_3
```

Out[]:

```
The model 'PeftModelForCausalLM' is not supported for text-generation. Supported models are
['BartForCausalLM', 'BertLMHeadModel', 'BertGenerationDecoder', 'BigBirdForCausalLM',
'BigBirdPegasusForCausalLM', 'BioGptForCausalLM', 'BlenderbotForCausalLM',
'BlenderbotSmallForCausalLM', 'BloomForCausalLM', 'CamembertForCausalLM', 'LlamaForCausalLM',
'CodeGenForCausalLM', 'CohereForCausalLM', 'CpmAntForCausalLM', 'CTRLLMHeadModel',
'Data2VecTextForCausalLM', 'DbrxForCausalLM', 'ElectraForCausalLM', 'ErnieForCausalLM',
```

```
'FalconForCausalLM', 'FuyuForCausalLM', 'GemmaForCausalLM', 'Gemma2ForCausalLM',
'GitForCausalLM', 'GPT2LMHeadModel', 'GPT2LMHeadModel', 'GPTBigCodeForCausalLM',
'GPTNeoForCausalLM', 'GPTNeoXForCausalLM', 'GPTNeoXJapaneseForCausalLM', 'GPTJForCausalLM',
'JambaForCausalLM', 'JetMoeForCausalLM', 'LlamaForCausalLM', 'MambaForCausalLM',
'Mamba2ForCausalLM', 'MarianForCausalLM', 'MBartForCausalLM', 'MegaForCausalLM',
'MegatronBertForCausalLM', 'MistralForCausalLM', 'MixtralForCausalLM', 'MptForCausalLM',
'MusicgenForCausalLM', 'MusicgenMelodyForCausalLM', 'MvpForCausalLM', 'NemotronForCausalLM',
'OlmoForCausalLM', 'OpenLlamaForCausalLM', 'OpenAIGPTLMHeadModel', 'OPTForCausalLM',
'PegasusForCausalLM', 'PersimmonForCausalLM', 'PhiForCausalLM', 'Phi3ForCausalLM',
'PLBartForCausalLM', 'ProphetNetForCausalLM', 'QDQBertLMHeadModel', 'Qwen2ForCausalLM',
'Qwen2MoeForCausalLM', 'RecurrentGemmaForCausalLM', 'ReformerModelWithLMHead',
'RemBertForCausalLM', 'RobertaForCausalLM', 'RobertaPreLayerNormForCausalLM',
'RoCBertForCausalLM', 'RoFormerForCausalLM', 'RwkvForCausalLM', 'Speech2Text2ForCausalLM',
'StableLmForCausalLM', 'Starcoder2ForCausalLM', 'TransfoXLLMHeadModel', 'TrOCRForCausalLM',
'WhisperForCausalLM', 'XGLMForCausalLM', 'XLMWithLMHeadModel', 'XLMProphetNetForCausalLM',
'XLMRobertaForCausalLM', 'XLMRobertaXLForCausalLM', 'XLNetLMHeadModel', 'XmodForCausalLM'].
```

Out[]:

```
Running inference on 2,816 test samples…
  Processed 2,720 / 2,816

═══════════════════════════════════════════════════════════
  📊  Evaluation Results — Gemma-2-9B-IT
═══════════════════════════════════════════════════════════
  Accuracy         : 0.2024
  Weighted F1      : 0.3097
  Macro F1         : 0.3050
───────────────────────────────────────────────────────────
              precision    recall  f1-score   support

    Remember       0.88      0.80      0.83       133
  Understand       0.97      0.25      0.40       762
       Apply       0.95      0.14      0.24       761
     Analyze       1.00      0.06      0.12       347
    Evaluate       1.00      0.06      0.12       370
      Create       0.92      0.27      0.42       443

   micro avg       0.94      0.20      0.33      2816
   macro avg       0.95      0.26      0.36      2816
weighted avg       0.96      0.20      0.31      2816
```

In []:

```
# ── 7.9  Free VRAM ──────────────────────────────────────
del model_3, trainer_3, tokenizer_3
gc.collect()
torch.cuda.empty_cache()
print("✅ VRAM freed.")
```

Out[]:

```
✅ VRAM freed.
```

8 · Cross-Model Comparison Summary

In []:
```
# ──────────────────────────────────────────────────────────────
#  Aggregate results from all three models into a clean table
# ──────────────────────────────────────────────────────────────

summary_rows = []
for model_name, res in ALL_RESULTS.items():
    summary_rows.append({
        "Model"          : model_name,
        "Accuracy"       : res["accuracy"],
        "Weighted F1"    : res["f1_weighted"],
        "Macro F1"       : res["f1_macro"],
    })

summary_df = pd.DataFrame(summary_rows).set_index("Model")
summary_df = summary_df.sort_values("Weighted F1", ascending=False)

print("\n" + "="*60)
print("   📋  Final Comparison — All Models")
print("="*60)
print(summary_df.to_string())
print("="*60)
best = summary_df.index[0]
print(f"\n🏆  Best model by Weighted F1: {best}  ({summary_df.loc[best, 'Weighted F1']:.4f})")
```

Out[]:
```
============================================================
   📋  Final Comparison — All Models
============================================================
                         Accuracy  Weighted F1  Macro F1
Model
Mistral-7B-Instruct-v0.3    0.9322       0.9324    0.9206
Qwen2-7B-Instruct           0.9247       0.9250    0.9149
Gemma-2-9B-IT               0.2024       0.3097    0.3050
============================================================

🏆  Best model by Weighted F1: Mistral-7B-Instruct-v0.3  (0.9324)
```

In []:
```
# ──────────────────────────────────────────────────────────────
#  Save full results to JSON for downstream analysis
# ──────────────────────────────────────────────────────────────

save_path = "./bloom_finetuning_results.json"

serialisable = {
    model: {
        k: v for k, v in res.items()
        if k not in ("predictions", "ground_truth")   # omit large lists
    }
    for model, res in ALL_RESULTS.items()
}
```

```
with open(save_path, "w") as f:
    json.dump(serialisable, f, indent=2)

print(f"✅ Results saved to: {save_path}")
print(json.dumps(serialisable, indent=2))
```

Out[]:

```
✅ Results saved to: ./bloom_finetuning_results.json
{
  "Mistral-7B-Instruct-v0.3": {
    "model": "Mistral-7B-Instruct-v0.3",
    "accuracy": 0.9322,
    "f1_weighted": 0.9324,
    "f1_macro": 0.9206
  },
  "Qwen2-7B-Instruct": {
    "model": "Qwen2-7B-Instruct",
    "accuracy": 0.9247,
    "f1_weighted": 0.925,
    "f1_macro": 0.9149
  },
  "Gemma-2-9B-IT": {
    "model": "Gemma-2-9B-IT",
    "accuracy": 0.2024,
    "f1_weighted": 0.3097,
    "f1_macro": 0.305
  }
}
```

In []:

```
# ──────────────────────────────────────────────────────────
#  CSV 1 — All predictions from all 3 models
# ──────────────────────────────────────────────────────────

prediction_rows = []

for model_name, res in ALL_RESULTS.items():
    for i, (pred, truth) in enumerate(zip(res["predictions"], res["ground_truth"])):
        prediction_rows.append({
            "sample_id"   : i,
            "model"       : model_name,
            "ground_truth": truth,
            "prediction"  : pred,
            "correct"     : int(pred == truth),
        })

predictions_df = pd.DataFrame(prediction_rows)

predictions_csv_path = "./bloom_all_predictions.csv"
predictions_df.to_csv(predictions_csv_path, index=False)

print(f"✅ Predictions CSV saved → {predictions_csv_path}")
print(f"   Rows    : {len(predictions_df):,}")
print(f"   Columns : {list(predictions_df.columns)}")
print(predictions_df.head(6).to_string(index=False))
```

Out[]:

```
✅ Predictions CSV saved → ./bloom_all_predictions.csv
   Rows    : 8,448
   Columns : ['sample_id', 'model', 'ground_truth', 'prediction', 'correct']
 sample_id                   model ground_truth prediction  correct
         0 Mistral-7B-Instruct-v0.3   Understand Understand        1
         1 Mistral-7B-Instruct-v0.3     Evaluate   Evaluate        1
         2 Mistral-7B-Instruct-v0.3   Understand Understand        1
         3 Mistral-7B-Instruct-v0.3   Understand Understand        1
         4 Mistral-7B-Instruct-v0.3     Evaluate   Evaluate        1
         5 Mistral-7B-Instruct-v0.3      Analyze    Analyze        1
```

In []:

```python
# ──────────────────────────────────────────────────────────────
#  CSV 2 — Class-wise metrics for all 3 models
#  Columns: model | class | precision | recall | f1 | support
# ──────────────────────────────────────────────────────────────

from sklearn.metrics import classification_report

classwise_rows = []

for model_name, res in ALL_RESULTS.items():
    report = classification_report(
        res["ground_truth"],
        res["predictions"],
        labels=BLOOM_LABELS,
        output_dict=True,
        zero_division=0,
    )

    # Per-class rows
    for label in BLOOM_LABELS:
        if label in report:
            classwise_rows.append({
                "model"    : model_name,
                "class"    : label,
                "precision": round(report[label]["precision"], 4),
                "recall"   : round(report[label]["recall"],    4),
                "f1_score" : round(report[label]["f1-score"],  4),
                "support"  : int(report[label]["support"]),
                "row_type" : "per_class",
            })

    # Aggregate rows (macro avg, weighted avg, accuracy)
    for agg_key in ["macro avg", "weighted avg"]:
        classwise_rows.append({
            "model"    : model_name,
            "class"    : agg_key,
            "precision": round(report[agg_key]["precision"], 4),
            "recall"   : round(report[agg_key]["recall"],    4),
            "f1_score" : round(report[agg_key]["f1-score"],  4),
            "support"  : int(report[agg_key]["support"]),
            "row_type" : "aggregate",
        })
```

```
    classwise_rows.append({
        "model"    : model_name,
        "class"    : "accuracy",
        "precision": "",
        "recall"   : "",
        "f1_score" : round(res["accuracy"], 4),
        "support"  : len(res["ground_truth"]),
        "row_type" : "aggregate",
    })

classwise_df = pd.DataFrame(classwise_rows)

classwise_csv_path = "./bloom_classwise_metrics.csv"
classwise_df.to_csv(classwise_csv_path, index=False)

print(f"✅ Class-wise metrics CSV saved → {classwise_csv_path}")
print(f"   Rows    : {len(classwise_df):,}")
print(f"   Columns : {list(classwise_df.columns)}")
print()
print(classwise_df.to_string(index=False))
```

Out[]:

```
✅ Class-wise metrics CSV saved → ./bloom_classwise_metrics.csv
   Rows    : 27
   Columns : ['model', 'class', 'precision', 'recall', 'f1_score', 'support', 'row_type']
```

model	class	precision	recall	f1_score	support	row_type
Mistral-7B-Instruct-v0.3	Remember	0.8478	0.8797	0.8635	133	per_class
Mistral-7B-Instruct-v0.3	Understand	0.9639	0.9449	0.9543	762	per_class
Mistral-7B-Instruct-v0.3	Apply	0.9417	0.9343	0.9380	761	per_class
Mistral-7B-Instruct-v0.3	Analyze	0.9217	0.9164	0.9191	347	per_class
Mistral-7B-Instruct-v0.3	Evaluate	0.9548	0.9135	0.9337	370	per_class
Mistral-7B-Instruct-v0.3	Create	0.8826	0.9503	0.9152	443	per_class
Mistral-7B-Instruct-v0.3	macro avg	0.9188	0.9232	0.9206	2816	aggregate
Mistral-7B-Instruct-v0.3	weighted avg	0.9332	0.9322	0.9324	2816	aggregate
Mistral-7B-Instruct-v0.3	accuracy			0.9322	2816	aggregate
Qwen2-7B-Instruct	Remember	0.8369	0.8872	0.8613	133	per_class
Qwen2-7B-Instruct	Understand	0.9693	0.9121	0.9398	762	per_class
Qwen2-7B-Instruct	Apply	0.9213	0.9382	0.9297	761	per_class
Qwen2-7B-Instruct	Analyze	0.8939	0.9222	0.9078	347	per_class
Qwen2-7B-Instruct	Evaluate	0.9654	0.9054	0.9344	370	per_class
Qwen2-7B-Instruct	Create	0.8828	0.9526	0.9164	443	per_class
Qwen2-7B-Instruct	macro avg	0.9116	0.9196	0.9149	2816	aggregate
Qwen2-7B-Instruct	weighted avg	0.9267	0.9247	0.9250	2816	aggregate
Qwen2-7B-Instruct	accuracy			0.9247	2816	aggregate
Gemma-2-9B-IT	Remember	0.876	0.797	0.8346	133	per_class
Gemma-2-9B-IT	Understand	0.9746	0.252	0.4004	762	per_class
Gemma-2-9B-IT	Apply	0.9464	0.1393	0.2428	761	per_class
Gemma-2-9B-IT	Analyze	1.0	0.0634	0.1192	347	per_class
Gemma-2-9B-IT	Evaluate	1.0	0.0622	0.1170	370	per_class
Gemma-2-9B-IT	Create	0.9167	0.2731	0.4209	443	per_class
Gemma-2-9B-IT	macro avg	0.9523	0.2645	0.3558	2816	aggregate
Gemma-2-9B-IT	weighted avg	0.9597	0.2024	0.3097	2816	aggregate
Gemma-2-9B-IT	accuracy			0.2024	2816	aggregate

---

9 · Inference Demo — Best Model

Load the saved best adapter and classify new learning outcomes interactively.

In []:

```
# ──────────────────────────────────────────────────────────────
#  Map model label → saved directory & HuggingFace model ID
# ──────────────────────────────────────────────────────────────

MODEL_MAP = {
    MODEL_1_LABEL: (MODEL_1_NAME, MODEL_1_OUTDIR),
    MODEL_2_LABEL: (MODEL_2_NAME, MODEL_2_OUTDIR),
    MODEL_3_LABEL: (MODEL_3_NAME, MODEL_3_OUTDIR),
}

best_base_id, best_adapter_dir = MODEL_MAP[best]

print(f"Loading best model : {best}")
```

```
print(f"  Base model       : {best_base_id}")
print(f"  Adapter dir      : {best_adapter_dir}")
```

Out[]:

```
Loading best model : Mistral-7B-Instruct-v0.3
  Base model       : mistralai/Mistral-7B-Instruct-v0.3
  Adapter dir      : ./mistral_bloom_finetuned
```

In []:

```
# ── Load base + adapter for inference ──────────────────────
bnb_inf = BitsAndBytesConfig(
    load_in_4bit              = True,
    bnb_4bit_use_double_quant = True,
    bnb_4bit_quant_type       = "nf4",
    bnb_4bit_compute_dtype    = torch.bfloat16,
)

base_inf = AutoModelForCausalLM.from_pretrained(
    best_base_id,
    quantization_config = bnb_inf,
    device_map          = "auto",
    trust_remote_code   = True,
)

model_inf     = PeftModel.from_pretrained(base_inf, best_adapter_dir)
tokenizer_inf = AutoTokenizer.from_pretrained(best_adapter_dir, trust_remote_code=True)

print(f"✅ {best} loaded for inference.")
```

Out[]:

```
Loading checkpoint shards:   0%|          | 0/3 [00:00<?, ?it/s]
```

Out[]:

```
✅ Mistral-7B-Instruct-v0.3 loaded for inference.
```

In []:

```
# ── Interactive inference ──────────────────────────────────

def classify_learning_outcome(learning_outcome: str) -> str:
    prompt = make_prompt(learning_outcome)
    inputs = tokenizer_inf(prompt, return_tensors="pt").to(model_inf.device)
    with torch.no_grad():
        output = model_inf.generate(
            **inputs,
            max_new_tokens = 8,
            do_sample      = False,
            pad_token_id   = tokenizer_inf.eos_token_id,
        )
    generated = tokenizer_inf.decode(output[0], skip_special_tokens=True)
    answer    = generated.split("### Bloom's Level:")[-1].strip().split()[0].capitalize()
    return answer if answer in BLOOM_LABELS else "Unknown"


# ── Sample predictions ─────────────────────────────────────
demo_outcomes = [
    "Define the key terms used in thermodynamics.",
```

```
    "Explain the principles of natural selection.",
    "Calculate the acceleration of a falling object using Newton's second law.",
    "Compare and contrast supervised and unsupervised machine learning.",
    "Critique a peer's experimental design for potential confounds.",
    "Design an algorithm to solve the travelling salesman problem.",
]

print(f"{'−'*65}")
print(f"{'Learning Outcome':<50}  {'Predicted Level'}")
print(f"{'−'*65}")
for lo in demo_outcomes:
    pred = classify_learning_outcome(lo)
    print(f"{lo[:50]:<50}  {pred}")
print(f"{'−'*65}")
```

Out[]:

```
_________________________________________________________________
Learning Outcome                                   Predicted Level
_________________________________________________________________

Define the key terms used in thermodynamics.       Understand
Explain the principles of natural selection.       Understand
Calculate the acceleration of a falling object usi Apply
Compare and contrast supervised and unsupervised m Evaluate
Critique a peer's experimental design for potentia Evaluate
Design an algorithm to solve the travelling salesm Create
_________________________________________________________________
```